

# Lesson 2 · How a Computer Stores Things in Memory

Course: Graphics with C++

Level: Absolute beginner

Est. 50–70 min

In Lesson 1 we kept using one slippery word: the **framebuffer** is "a region of memory." This lesson opens that word up. Every image, every 3D model, and every single variable your program touches lives in **memory** — so understanding what memory really is, why we use it, and what kinds exist is the foundation under all of graphics and, honestly, all of programming.

We'll stay concrete: real diagrams, a small C++ program with its output shown, and every term defined the first time it appears.

## 1. What "memory" actually is

Picture an impossibly long street of tiny lockers. The lockers are numbered `0, 1, 2, 3, ...` all the way up. Each locker holds exactly one **byte** (8 bits — recall from Lesson 1 that's a value from 0 to 255). That street is your computer's memory.

### THE ONE IDEA TO HOLD ONTO

Memory is one enormous, numbered sequence of byte-sized slots. To store something you write bytes into some slots; to read it back you go to those slot numbers and read. That's the whole model.

The number of a slot is its **address**.

### JARGON · ADDRESS

An **address** is the position of a byte in memory — just an integer, like a house number on that street. When the CPU wants a value, it asks memory for "the bytes starting at address N."

Memory: one long line of numbered, byte-sized slots



*Each slot is one byte with its own address. A value that needs more than one byte (like a 4-byte int) simply occupies several consecutive slots.*

## 2. How a single value is stored: binary

The slots don't hold decimal numbers or letters directly — at the hardware level everything is **bits**, each a single 0 or 1 (a tiny switch that's off or on). A byte is 8 of those switches. To store the number 42, the computer picks the pattern of 8 bits that *adds up* to 42.

Each of the 8 positions has a "place value" — like columns in ordinary decimal, but doubling each step instead of  $\times 10$ :

|     |    |    |    |   |   |   |   |
|-----|----|----|----|---|---|---|---|
| 128 | 64 | 32 | 16 | 8 | 4 | 2 | 1 |
| 0   | 0  | 1  | 0  | 1 | 0 | 1 | 0 |

Turned-on bits:  $32 + 8 + 2 = 42$

So the byte for 42 is the bit pattern 00101010.

*One byte = 8 bits with place values 128...1. The number 42 is the pattern 00101010 (32 + 8 + 2).*

### JARGON · BINARY

**Binary** is counting with only two digits, 0 and 1 (base 2), because a bit can only be off or on. Decimal columns are 1, 10, 100...; binary columns are 1, 2, 4, 8, 16... Everything in a computer — numbers, text, images, code — is ultimately stored as binary.

One byte can only reach 255 (all eight bits on =  $128+64+\dots+1$ ). Bigger numbers simply use **more bytes**. That's exactly why different kinds of data reserve different amounts of memory.

## 3. Data types and how many bytes they take

In C++ you tell the compiler what *kind* of value a variable holds — its **type** — and the type decides how many bytes get reserved. More bytes means a bigger range (for integers) or more precision (for decimals).

Here are the common types and their sizes **on your Mac** (an arm64 / Apple Silicon machine — these are typical for any 64-bit computer, but the C++ standard lets them vary, so never assume; measure):

| Type                          | Meaning                   | Size on your Mac |
|-------------------------------|---------------------------|------------------|
| <code>char</code>             | one character / one byte  | 1 byte           |
| <code>bool</code>             | true or false             | 1 byte           |
| <code>int</code>              | whole number              | 4 bytes          |
| <code>float</code>            | decimal number            | 4 bytes          |
| <code>double</code>           | decimal, double precision | 8 bytes          |
| <code>int*</code> (a pointer) | an address                | 8 bytes          |

## JARGON · FLOAT AND DOUBLE

**float** and **double** store numbers with a fractional part (like 3.14) using *floating point* — a scientific-notation-like format. **double** uses twice the bytes of a **float**, so it holds more digits of precision. Graphics math uses these constantly for positions, colors, and directions.

A 4-byte `int` has 32 bits, so it can represent  $2^{32} \approx$  **4.29 billion** different values. That range is split between negative and positive, giving roughly  $-2.1$  billion to  $+2.1$  billion.

### Example: a program that reveals sizes and addresses

Here's a short program that asks C++ directly for these sizes, and also prints where two variables live in memory:

```
#include <iostream>

int main() {
    // sizeof tells you how many bytes a type occupies on THIS machine.
    std::cout << "char:    " << sizeof(char)    << " byte(s)\n";
    std::cout << "bool:    " << sizeof(bool)    << " byte(s)\n";
    std::cout << "int:     " << sizeof(int)     << " byte(s)\n";
    std::cout << "float:   " << sizeof(float)  << " byte(s)\n";
    std::cout << "double:  " << sizeof(double) << " byte(s)\n";
    std::cout << "pointer: " << sizeof(int*)   << " byte(s)\n";

    int a = 42;
    int b = 7;

    // The & operator asks: "what address does this variable live at?"
    std::cout << "a = " << a << ", stored at address " << &a << '\n';
    std::cout << "b = " << b << ", stored at address " << &b << '\n';
    return 0;
}
```

On this arm64 Mac, that program prints (the addresses are different on every run — that's normal):

```
char:    1 byte(s)
bool:    1 byte(s)
int:     4 byte(s)
float:   4 byte(s)
double:  8 byte(s)
pointer: 8 byte(s)
a = 42, stored at address 0x16bb3a4a8
b = 7, stored at address 0x16bb3a4a4
```

#### JARGON · SIZEOF AND THE & OPERATOR

`sizeof(T)` is a built-in that reports how many bytes type `T` takes. The **address-of operator** `&x` gives the memory address where variable `x` lives. Notice the two addresses above differ by 4 — the two 4-byte ints sit right next to each other in memory.

## 4. Addresses and pointers

Because every variable has an address, a program can hand around *the address itself* instead of copying the whole value. A variable that stores an address is called a **pointer**.

#### JARGON · POINTER

A **pointer** is a variable whose value is a memory address — it "points at" another piece of data. Written `int*` ("pointer to int"). This is how a program refers to a big block of memory (like an image's pixels) cheaply: pass the one small address, not a copy of all the bytes.

#### WHY THIS MATTERS FOR GRAPHICS

An image can be millions of bytes. You never want to copy that around by accident. Pointers (and their friends, arrays and references) let your code say "the pixels start *here*" with a single 8-byte address. Every graphics API you'll ever touch passes image and buffer data by address.

## 5. Why keep things in memory at all? Memory vs storage

Here's the crucial fact: **the CPU can only compute on data that's in memory**. More precisely, it loads bytes from main memory into tiny on-chip slots called **registers**, does arithmetic there, and writes results back to memory. It cannot add two numbers that are sitting on your SSD — they must be brought into memory first.

So why not just use the disk for everything?

- **Main memory (RAM) is fast but forgetful.** It answers in tens of nanoseconds, but it's **volatile** — cut the power and its contents vanish.
- **Disk/SSD is slow but permanent.** It's thousands of times slower to reach, but **non-volatile** — your files survive a reboot.

#### JARGON · VOLATILE VS NON-VOLATILE

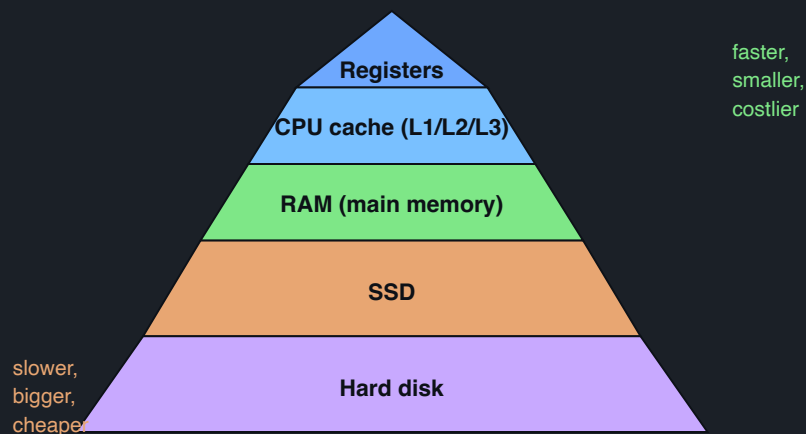
**Volatile** memory loses everything when powered off (RAM, caches, registers). **Non-volatile** storage keeps its contents without power (SSD, hard disk, flash drives). "Saving a file" means copying from volatile memory to non-volatile storage so it outlives the program.

## THE DIVISION OF LABOR

Data you're actively working on lives in **RAM** (fast, temporary). Data you want to keep lives on **disk** (slow, permanent). Your gradient from Lesson 1 was computed in memory, then saved to disk as a `.ppm` file so it wouldn't disappear.

## 6. The types of memory: the memory hierarchy

If RAM is "fast" and disk is "slow," reality is actually a whole ladder of memory, each rung faster and smaller than the one below. Faster memory is more expensive per byte, so computers use a little of the fast stuff and a lot of the slow stuff, arranged as a hierarchy.



*The memory hierarchy. The CPU checks the fast top layers first; only what doesn't fit falls through to the slower, larger layers below.*

Rough figures (they vary a lot by hardware — treat them as orders of magnitude, not exact):

| Memory        | Typical size                | Approx. time to reach | Volatile? |
|---------------|-----------------------------|-----------------------|-----------|
| Registers     | a few hundred bytes         | under 1 ns            | yes       |
| L1 cache      | tens of KB                  | ~1 ns                 | yes       |
| L2 / L3 cache | hundreds of KB – tens of MB | ~4–20 ns              | yes       |
| RAM           | 8 – 64 GB                   | ~80–100 ns            | yes       |
| SSD           | 256 GB – several TB         | ~50–150 $\mu$ s       | no        |
| Hard disk     | 1 – 20 TB                   | ~5–10 ms              | no        |

### JARGON · REGISTER, CACHE, RAM

A **register** is a handful of bytes physically inside the CPU where actual computation happens — the fastest memory there is. A **cache** is a small, fast memory that keeps recently-used data close to the CPU so it doesn't have to make the slow trip to RAM every time. **RAM** stands for *Random Access Memory*: "random access" means any address can be read in about the same time, in any order — unlike a tape you'd have to wind through.

## GRAPHICS FOOTNOTE: THE GPU HAS ITS OWN MEMORY

A **GPU** (graphics processor) traditionally has its own separate memory called **VRAM**, and images/textures must be copied into it before the GPU can draw with them. Your Apple Silicon Mac is special here: it uses **unified memory**, where the CPU and GPU share one pool of RAM — so there's no separate copy step. We'll care about this a lot once we reach the GPU.

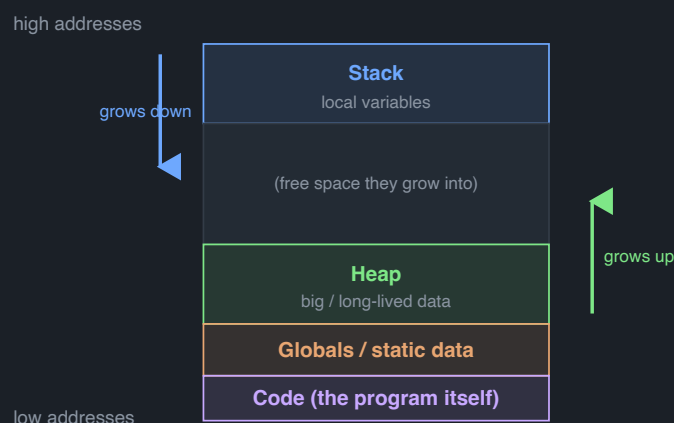
## 7. A quick note on RAM vs ROM

You'll also hear **ROM** — *Read-Only Memory*. Unlike RAM, it's non-volatile and normally not written to during use; it holds things like the firmware that starts your computer. The RAM in your machine is technically **DRAM** (dynamic), which is dense and cheap; caches are **SRAM** (static), which is faster but takes more space per byte. You don't need these acronyms to write graphics code — just know that "memory" isn't one single thing.

## 8. How your program's memory is organized: stack and heap

When your program runs, the operating system hands it a big span of addresses to use. That span is split into regions with different jobs. Two of them matter constantly:

- The **stack** — where local variables (like `a` and `b` above) live. It's automatic: space is reserved when a function starts and freed the instant the function returns. Fast, but limited in size.
- The **heap** — a large pool you request memory from explicitly when you need something big or long-lived (like a whole image). You control when it's allocated and freed.



*A running program's memory. The stack grows downward from high addresses; the heap grows upward toward it. Globals and the program's own code sit at the bottom.*

### JARGON · STACK, HEAP, ALLOCATION

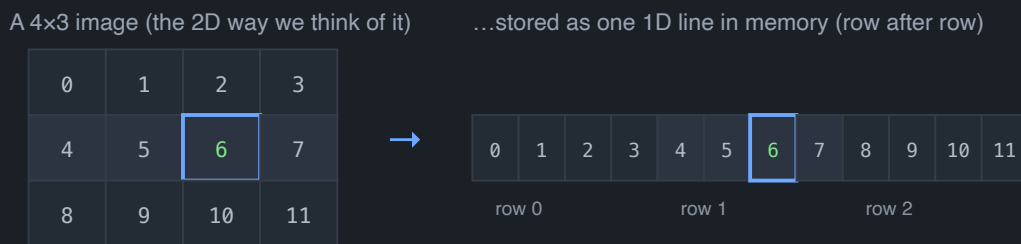
The **stack** holds a function's local variables and is managed automatically (last-in, first-out). The **heap** is memory you request on demand for data whose size or lifetime you decide at run time. **Allocation** is the act of reserving memory; **freeing** (or *deallocation*) is giving it back. Forgetting to free heap memory is a **memory leak**.

### WATCH OUT

The stack is small (often just a few megabytes). A full-screen image is far too big to put there — try, and you get a *stack overflow* crash. Big buffers like images belong on the heap. In modern C++ you rarely call raw allocation yourself; containers like `std::vector` manage heap memory for you, which is exactly what we'll use for an image buffer.

## 9. Back to graphics: how an image sits in memory

Memory is one-dimensional — a single line of bytes — but an image is two-dimensional. So how does a 2D grid of pixels fit into 1D memory? We simply lay the rows out end to end: all of row 0, then all of row 1, then row 2, and so on. This is called **row-major** order.



`index = y * width + x`

Pixel (x=2, y=1) in a width-4 image →  $1 \times 4 + 2 = \text{index } 6$  (highlighted).

*A 2D image flattened into 1D memory, row by row. The formula `y * width + x` converts a pixel's (x, y) into its position in that single line.*

That one formula, `index = y * width + x`, is how essentially every framebuffer is addressed. (When each pixel is 3 bytes for R, G, B, you multiply by 3: the red byte of pixel (x, y) sits at `(y * width + x) * 3`.)

### HOW MUCH MEMORY IS AN IMAGE?

Multiply width × height × bytes-per-pixel. A 1024×1024 image with 3 bytes per pixel is  $1024 \times 1024 \times 3 = \mathbf{3,145,728 \text{ bytes} \approx 3 \text{ MB}}$ . A 1920×1080 photo is about **6 MB**. That's why images go on the heap, and why fast memory matters: the GPU may touch millions of these bytes every frame.

## 10. More worked examples

These tie the ideas together with concrete numbers.

### A · Storing the letter A

A `char` is one byte. Text is stored by mapping each character to a number using a code called **ASCII**. The letter `A` is 65, and 65 in binary is `01000001` (64 + 1). So a `char` holding `A` is literally the byte `01000001`. Lowercase `a` is 97 (`01100001`).

## B · When a value doesn't fit in its bytes

One byte can hold at most 255. The number 200 fits fine: `11001000` (128 + 64 + 8). But 300 needs a 9th bit (the 256 place), so it **cannot** live in a single byte — it would *overflow*. This is why a count that might pass 255 is stored in a wider type like `int` (4 bytes) instead of a single byte.

## C · How much memory an image needs

The size is always `width × height × bytes-per-pixel` (3 bytes per pixel for RGB):

| Image                 | Pixels    | Bytes (× 3) | ≈        |
|-----------------------|-----------|-------------|----------|
| 256 × 256             | 65,536    | 196,608     | ~192 KB  |
| 1920 × 1080 (Full HD) | 2,073,600 | 6,220,800   | ~5.9 MB  |
| 3840 × 2160 (4K)      | 8,294,400 | 24,883,200  | ~23.7 MB |

Notice how fast it grows: doubling width *and* height quadruples the memory.

## D · Finding a pixel inside the 1D buffer

Take a 256-pixel-wide RGB image and ask: where is the **red** byte of pixel (x = 10, y = 3)? Using `(y × width + x) × 3`:

```
(3 × 256 + 10) × 3
= (768 + 10) × 3
= 778 × 3
= 2334
```

So red is at index 2334, green at 2335, blue at 2336.

## E · Stack or heap for a photo?

A 4000 × 3000 photo at 3 bytes per pixel is 4000 × 3000 × 3 = 36,000,000 bytes ≈ **34 MB**. The stack is usually only a few MB, so putting 34 MB there would overflow it and crash. Therefore the image buffer must live on the **heap** (in practice, inside a `std::vector`). A loop counter or an (x, y) pair, being just a few bytes, happily stays on the stack.

# 11. What you learned

- Memory is **one long, numbered sequence of bytes**; each byte's number is its **address**.
- Everything is stored in **binary**; bigger values use more bytes, which is why each **type** reserves a fixed size (`int` = 4 bytes, `double` = 8, a pointer = 8 on your Mac).
- A **pointer** is a variable that holds an address — how programs refer to big data (like images) cheaply.
- The CPU computes only on data in memory; **RAM** is fast but **volatile**, **disk** is slow but **non-volatile** — hence "working data in RAM, saved files on disk."
- Memory is a **hierarchy**: registers → cache → RAM → SSD → disk, trading speed for size.



- A running program splits its memory into **stack** (small, automatic locals) and **heap** (big, on-demand data like images), plus globals and code.
- A 2D image lives in 1D memory in **row-major** order, addressed by `index = y * width + x`.
- New vocabulary: address, binary, bit/byte (recap), type sizes, `sizeof`, `&`, pointer, register, cache, RAM/ROM, volatile/non-volatile, memory hierarchy, stack, heap, allocation, memory leak, row-major.

If any part of this is fuzzy, tell me which section and I'll expand it and rebuild the PDF.